

Cassie_Duncan_Foundations_in_Data_Science_Final_Project

Cassandra Duncan

2025-10-12

Final Project Submission

Generate Summary Statistics

```
# read in the Sample RNA-seq Count Matrix
gene_df <- read.csv("/Users/cassieschool/Documents/Foundations_in_Data_Science/Final_Project/Data/count)

# Convert the data frame to long format from wide format
gene_df_long <- tidyr::pivot_longer(gene_df, cols = -X, names_to = "barcode", values_to = "Count")

# Subset data to only include one selected gene of interest (TACSTD2)
# Second Check-In update: omitted NA's from the beginning
one_gene <- na.omit(gene_df_long[gene_df_long$X == "ENSG00000184292.7", ])

# Generate full summary statistics (minimum, first quantile, median, mean, third quantile, and maximum)
summary(one_gene$Count)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##         1    9866   16523   19879   25916  119596
```

```
# Generate Standard Deviation separately since it is not included in the Summary Statistics
cat("Standard Deviation", "\n", sd(one_gene$Count), "\n")
```

```
## Standard Deviation
##  15161.47
```

```
# Summary statistics for second gene for final submission table
# Subset data to only include one selected gene of interest (CCND1)
# Second Check-In update: omitted NA's from the beginning
second_gene <- na.omit(gene_df_long[gene_df_long$X == "ENSG00000110092.4", ])

# Generate full summary statistics (minimum, first quantile, median, mean, third quantile, and maximum)
summary(second_gene$Count)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##       430    8316   16473   29649   33291  577835
```

```
# Generate Standard Deviation separately since it is not included in the Summary Statistics
cat("Standard Deviation", "\n", sd(second_gene$Count))
```

```
## Standard Deviation
## 44263
```

Create Histogram

```
library(ggplot2)

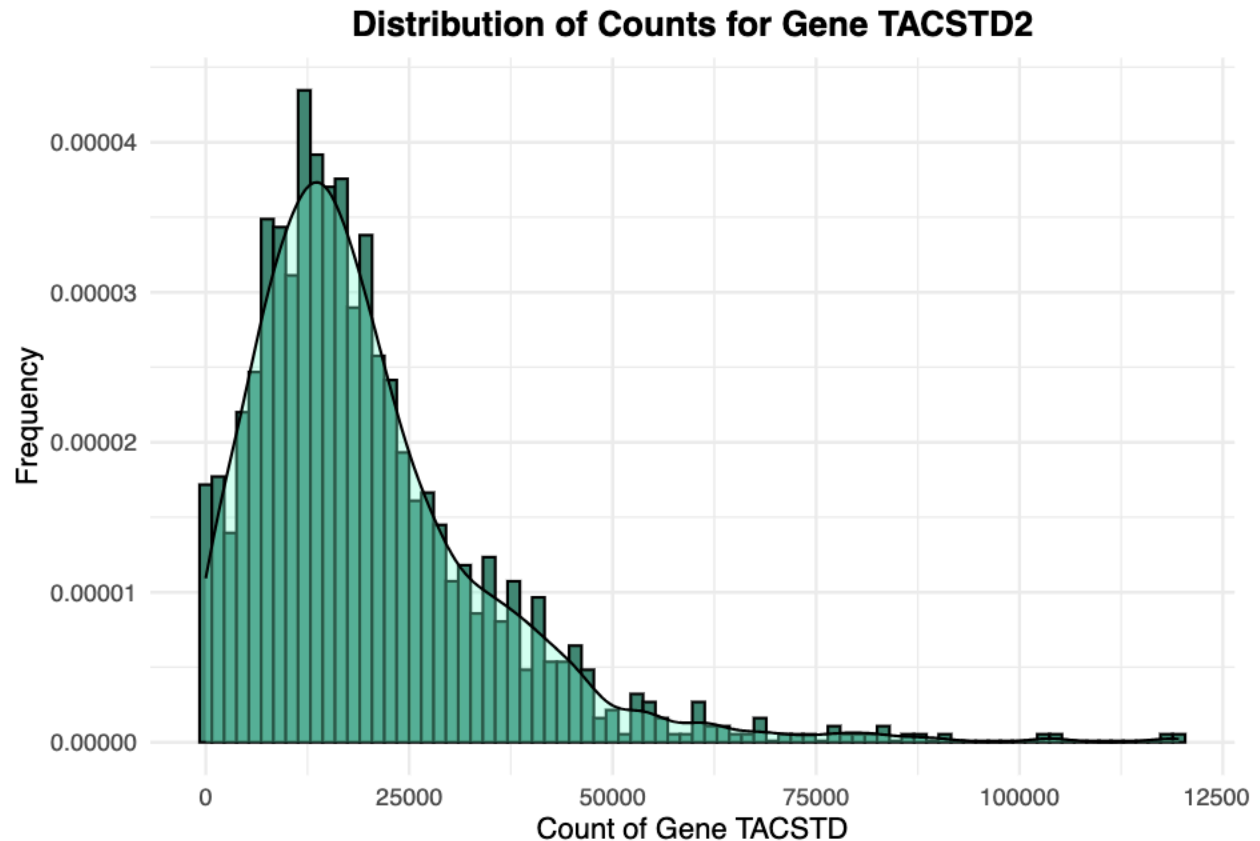
# remove scientific notation on y-axis from Samira's Discussion 1 feedback
options(scipen = 999)

# Create histogram of one gene of interest
gene_plot <- ggplot(one_gene, aes(x = Count)) +
  # Second Check-In: Got rid of binwidth and updated bins to lower value of 100 from Samira's discussion
  geom_histogram(bins = 80,
                 fill = "aquamarine4", #choose color of plot
                 aes(y = after_stat(density)), color = "black") +
  # Second Check-In: Add density curve layer inspired by Chayce
  geom_density(lwd = 0.5, fill = "aquamarine", alpha = 0.35) +
  # add titles to plot and axes
  labs(title = "Distribution of Counts for Gene TACSTD2",
       x = "Count of Gene TACSTD",
       y = "Frequency") +
  theme_minimal() +
  # bolded and centered title for second check-in
  theme(plot.title = element_text(face = "bold", hjust = 0.5))

# save above figure as .png file
png(filename = "/Users/cassieschool/Documents/Foundations_in_Data_Science/Final_Project/Data/gene_histogram.png")
# call above figure to save as .png
gene_plot
# close graphics to only save box plot generated above
dev.off()
```

```
## pdf
## 2
```

```
print(gene_plot)
```



Create Scatter Plot

```
# subset data for only two genes of interest
two_genes <- gene_df_long[gene_df_long$X == "ENSG00000184292.7" | gene_df_long$X == "ENSG00000110092.4"]

# convert the dataframe to wide format from long format so genes are column headers for count values
two_genes_wide <- tidyr::pivot_wider(two_genes, names_from = X, values_from = Count)

# Added for Second Check-In
# fit linear model
lin_model <- lm(ENSG00000184292.7 ~ ENSG00000110092.4, data = two_genes_wide)

# extract coefficients
coeffs <- coef(lin_model)
intercept <- coeffs[1]
slope <- coeffs[2]

# create equation label text
eq_label <- paste0("y = ",
                    round(slope, 2), "x + ",
                    round(intercept, 2))

# remove scientific notation on y-axis from Samira's Discussion 1 feedback
options(scipen = 999)

# Create scatterplot of comparing two selected genes
```

```

gene_scatter <- ggplot(two_genes_wide, aes(x = ENSG00000184292.7, y = ENSG00000110092.4)) +
  # set color and size of points in scatterplot
  # added some transparency to points for second check-in
  geom_point(alpha = 0.6, color = "darkorchid4", size = 0.5) +
  # add regression line to plot
  geom_smooth(method = "lm", se = TRUE, color = "seagreen3") +
  # add title to plot and axes
  labs(title = "Comparison of Counts for Gene TACSTD2 vs Gene CCND1",
        x = "Gene TACSTD2 Count",
        y = "Gene CCND1 Count") +
  # add equation annotation
  annotate("text", x = Inf, y = -Inf, label = eq_label,
           hjust = 1.1, vjust = -1.1, size = 3.5, color = "seagreen4") +
  theme_minimal() +
  # bolded and centered title for second check-in
  theme(plot.title = element_text(face = "bold", hjust = 0.5))

# save above figure as .png file
png(filename = "/Users/cassieschool/Documents/Foundations_in_Data_Science/Final_Project/Data/gene_scatter.png")
# call above figure to save as .png
gene_scatter

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```

# close graphics to only save box plot generated above
dev.off()

```

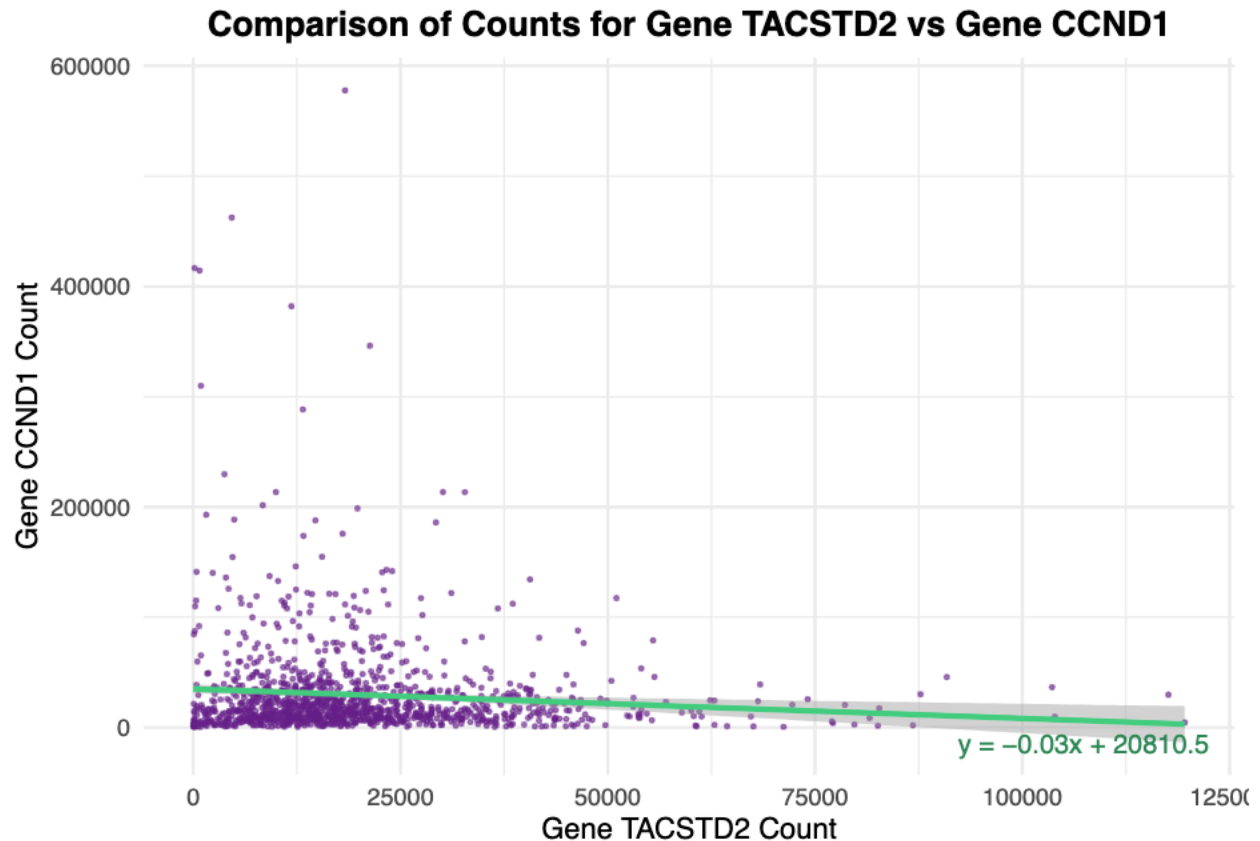
```

## pdf
## 2

```

```
print(gene_scatter)
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



Create Violin Plot

```
# reading in harrypotter library to use color scheme
library(harrypotter)

# read in metadata file
metadata <- read.csv("/Users/cassieschool/Documents/Foundations_in_Data_Science/Final_Project/Data/metadata.csv")

# subset metadata to only include one covariate of interest
covariate_metadata <- subset(metadata, select = c(barcode, paper_pathologic_stage))

# convert barcode values in metadata to have . instead of - so they can match up to data in count file
covariate_metadata$barcode <- gsub("-", ".", covariate_metadata$barcode)

# join count data and metadata by barcode/sample so gene counts can be compared to chosen covariate
gene_metadata <- merge(one_gene, covariate_metadata[, c("barcode", "paper_pathologic_stage")], by = "barcode")

# generate violin plot for distribution of count data based on covariate
# omitted NA values in plotting
gene_violin <- ggplot(na.omit(gene_metadata), aes(x = paper_pathologic_stage, y = Count, fill = paper_pathologic_stage)) +
  geom_violin() +
  # Second Check-In: add box plot to violin plot as I learned this was a standard from Discussion 1
  geom_boxplot(fill = "snow4", alpha = 0.4) +
  # add title to plot and axes
  labs(title = "Distribution of Count of Gene TACSTD2 for Disease Pathologic Stage",
       x = "Disease Pathologic Stage",
```

```

y = "Gene TACSTD2 Count",
# adding legend title as suggestion both from the professor and Samira from the first Check-In
fill = "Disease Pathologic Stage",) +
# Second Check-In: updated plot color scheme to harry potter Slytherin theme
scale_fill_hp_d(option = "Slytherin") +
theme_minimal() +
# bolded title for second check-in and made size smaller to fit on pdf
theme(plot.title = element_text(face = "bold", size = 12))

# save above figure as .png file
png(filename = "/Users/cassieschool/Documents/Foundations_in_Data_Science/Final_Project/Data/gene_violin",
# call above figure to save as .png
gene_violin
# close graphics to only save box plot generated above
dev.off()

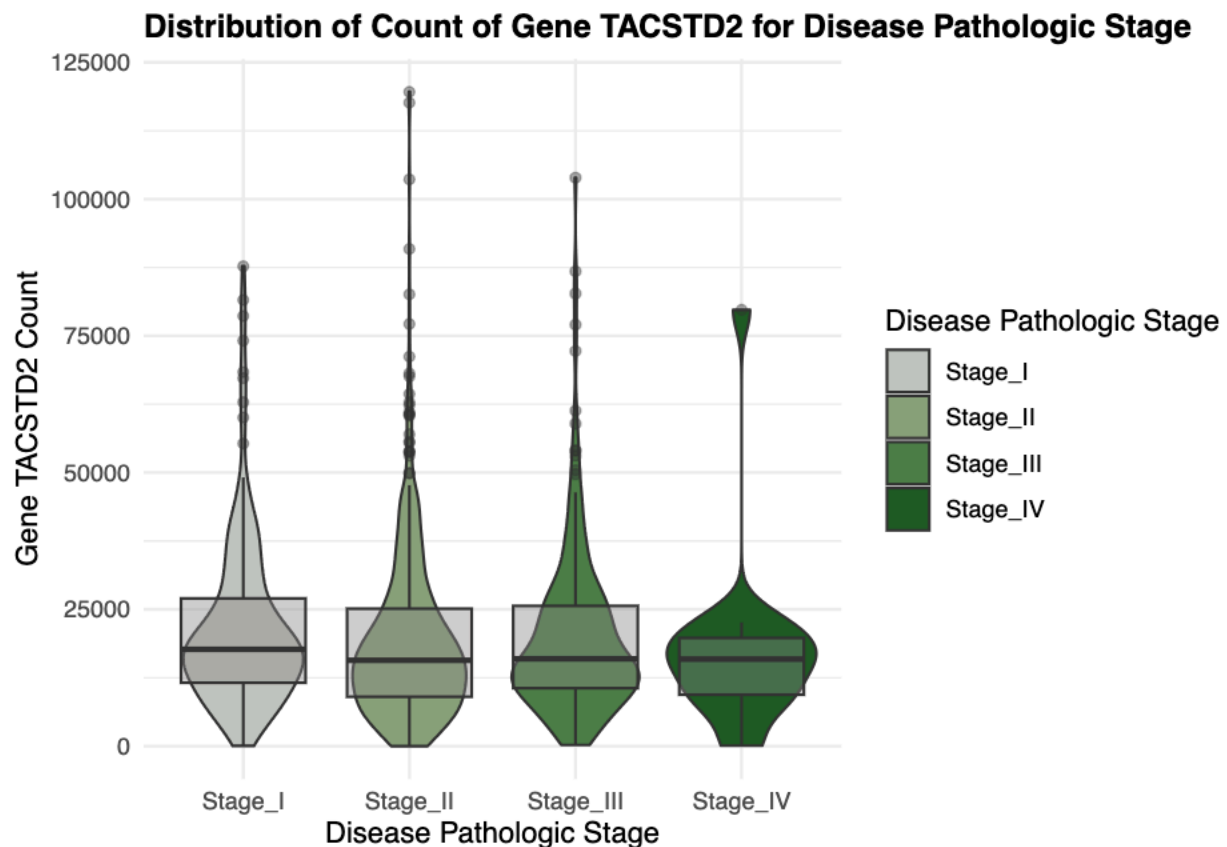
```

```

## pdf
## 2

```

```
print(gene_violin)
```



Create Heatmap Analysis

```
library(ComplexHeatmap)
```

```
## Loading required package: grid

## =====
## ComplexHeatmap version 2.24.1
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional
##   genomic data. Bioinformatics 2016.
##
##
## The new InteractiveComplexHeatmap package can directly export static
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(ComplexHeatmap))
## =====
```

```
library(circlize)
```

```
## =====
## circlize version 0.4.16
## CRAN page: https://cran.r-project.org/package=circlize
## Github page: https://github.com/jokergoo/circlize
## Documentation: https://jokergoo.github.io/circlize\_book/book/
##
## If you use it in published research, please cite:
## Gu, Z. circlize implements and enhances circular visualization
##   in R. Bioinformatics 2014.
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(circlize))
## =====
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```

# set first column in dataframe as row names
gene_df_new <- gene_df
rownames(gene_df_new) <- gene_df_new$X

## selecting 10 genes to use in heat map
ten_genes <- c("ENSG00000184292.7", "ENSG00000110092.4", "ENSG0000012048.23",
               "ENSG00000139618.16", "ENSG00000183765.23", "ENSG00000083093.10",
               "ENSG00000039068.19", "ENSG00000171862.11", "ENSG00000118046.16", "ENSG00000141510.18")

# keep only the 10 genes I want
ten_genes_subset <- subset(gene_df_new, rownames(gene_df_new) %in% ten_genes)

# there is definitely a more efficient way to go about this, but to name my genes by their actual gene
ten_genes_long <- tidyr::pivot_longer(ten_genes_subset, cols = -X, names_to = "barcode", values_to = "Count")

# then converted to wide format so gene names can be column names
ten_genes_wide <- tidyr::pivot_wider(ten_genes_long, names_from = X, values_from = Count)

# renamed my columns to their actual gene names instead of their ensembl ids
ten_genes_wide <- ten_genes_wide %>% rename(TACSTD2 = ENSG00000184292.7, CCND1 = ENSG00000110092.4, BRCA2 = ENSG00000139618.16, CHEK2 = ENSG00000183765.23, PALB1 = ENSG00000083093.10,
                                           CDH1 = ENSG00000039068.19, PTEN = ENSG00000171862.11, STK11 = ENSG00000118046.16)

# converted back to long format
ten_genes_long <- tidyr::pivot_longer(ten_genes_wide, cols = -barcode, names_to = "X", values_to = "Count")

# converted to wide format once again so sample names can be column headers
ten_genes_df <- tidyr::pivot_wider(ten_genes_long, names_from = barcode, values_from = Count)

# changed data set from tibble to data frame for heatmap generation
ten_genes_df <- as.data.frame(ten_genes_df)

# set first column as row names
rownames(ten_genes_df) <- ten_genes_df$X
# deleted first column that is no longer needed
ten_genes_df <- ten_genes_df[, -1]

# take earlier covariate metadata and do the same as above to set first column as row names and delete
covariate_metadata_new <- covariate_metadata
rownames(covariate_metadata_new) <- covariate_metadata_new$barcode
covariate_metadata_new <- covariate_metadata_new[, -1, drop=FALSE]

# Heatmap annotations
column_ha <- HeatmapAnnotation(
  Pathologic_Stage = covariate_metadata_new$paper_pathologic_stage,
  col = list(
    # Updated colors from second check-in
    Pathologic_Stage = c(Stage_I = "darkgreen", Stage_II = "forestgreen",
                        Stage_III = "limegreen", Stage_IV = "lawngreen")),
  annotation_legend_param = list(title = "Pathologic Stage")
)

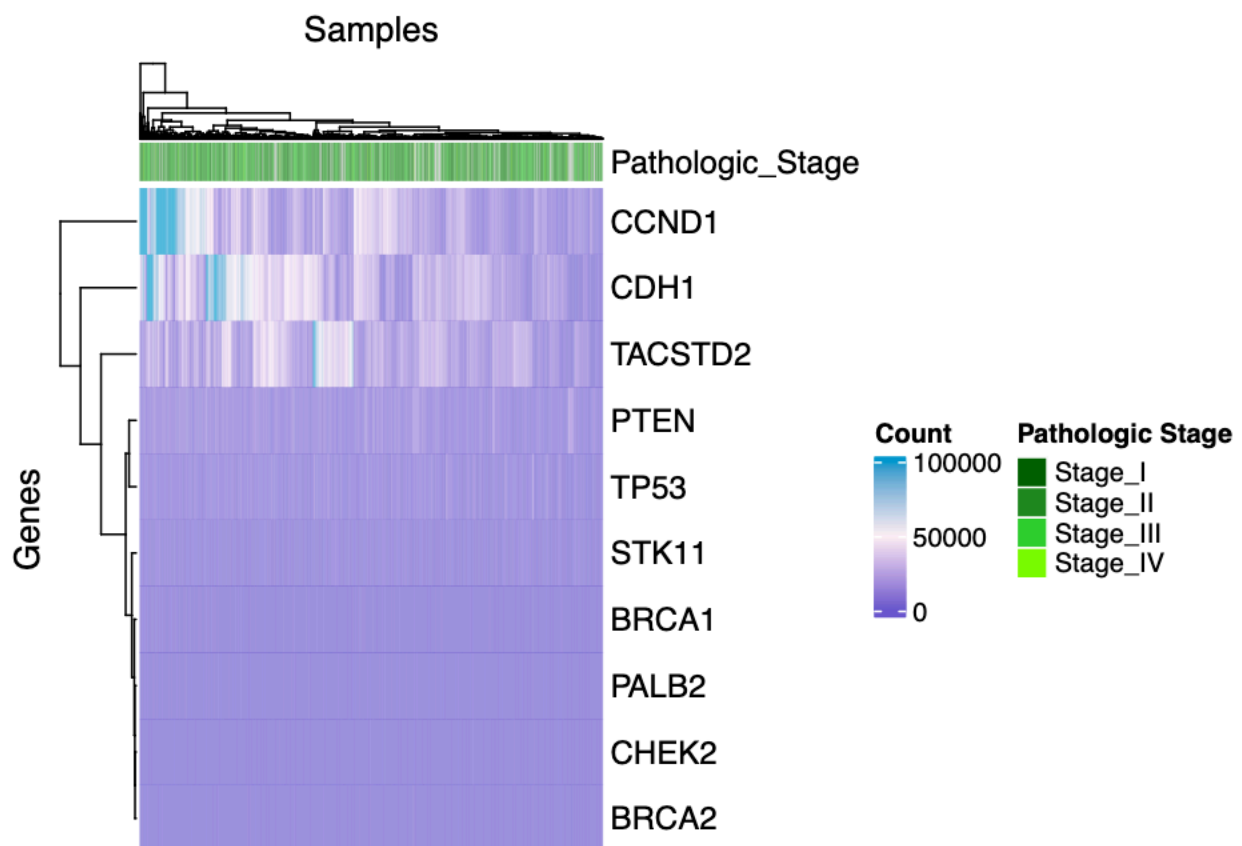
# add custom color palette for heat map after second check-in

```

```
heatmap_palette <- colorRamp2(c(min(ten_genes_df), 50000, 100000), c("slateblue3", "lavenderblush", "de
# Create heatmap
gene_heatmap <- Heatmap(
  ten_genes_df,
  col = heatmap_palette,
  name = "Counts",
  # added color from second check-in
  #col = heatmap_palette,
  top_annotation = column_ha,
  show_row_names = TRUE,
  show_column_names = FALSE,
  cluster_columns = TRUE,
  cluster_rows = TRUE,
  column_title = "Samples",
  row_title = "Genes",
  # updated legend title from second check-in feedback
  heatmap_legend_param = list(title = "Count")
)
```

Warning: The input is a data frame-like object, convert it to a matrix.

```
print(gene_heatmap)
```



```

# save above figure as .png file
png(filename = "/Users/cassieschool/Documents/Foundations_in_Data_Science/Final_Project/Data/gene_heatmap.png")
# added visual components from Second Check-In - Inspiration from Pritish
draw(gene_heatmap,
     column_title = "Heat Map of Gene Expression by Pathologic Stage",
     heatmap_legend_side = "right",
     annotation_legend_side = "right",
     merge_legend = TRUE)

# close graphics to only save box plot generated above
dev.off()

```

```

## pdf
## 2

```

Create Density Plot

```

# generate density plot
gene_density <- ggplot(na.omit(gene_metadata), aes(Count)) +
  # generate density plot by my covariate
  geom_density(aes(fill = paper_pathologic_stage), alpha = 0.4) +
  # add title to plot, axes, and legend
  labs(title = "Density of Counts for Gene TACSTD2 by Pathologic Stage",
       x = "Gene TACSTD2 Count",
       y = "Density",
       fill = "Pathologic Stage") +
  theme_minimal() +
  # choose color palette
  scale_fill_brewer(palette = "Dark2") +
  # bolded and centered title
  theme(plot.title = element_text(face = "bold", hjust = 0.5))

# save above figure as .png file
png(filename = "/Users/cassieschool/Documents/Foundations_in_Data_Science/Final_Project/Data/gene_density.png")
# call above figure to save as .png
gene_density
# close graphics to only save box plot generated above
dev.off()

```

```

## pdf
## 2

```

```

print(gene_density)

```

